Rapport Technique
Application d'Interpolation Numérique

Master 1 Génie Informatique — Cours d'Analyse Numérique

---

1. Présentation Générale

Cette application est développée avec **Streamlit**, un framework Python orienté data science, et constitue un outil pédagogique interactif dédié à l'étude comparative de deux méthodes d'interpolation numérique classiques :

- L'interpolation polynomiale de **Lagrange**
- L'interpolation par **spline quadratique**

Pour chaque méthode, l'application propose une double implémentation : l'une **symbolique** via la bibliothèque **SymPy**, l'autre **numérique rapide** via **SciPy**. Cette dualité permet d'analyser les différences en termes de précision, de lisibilité des résultats et de performance computationnelle.

---

2. Architecture du Projet

Le projet est organisé selon la structure suivante :

```
projet/
    app.py                  #   Point   d'entrée
principal                           (Streamlit)
    modules/
        lagrange.py           #   Fonctions   :
lagrange_sympy(),             lagrange_scipy()
        spline.py             #   Fonctions   :
spline_sympy(), spline_scipy(), calculer_zi()
```

Le fichier **app.py** orchestre l'ensemble de l'interface et délègue les calculs aux modules spécialisés *lagrange.py* et *spline.py*.

---

3. Fonctionnalités de l'Application

3.1 Saisie des Données (Barre Latérale)

L'utilisateur dispose de deux modes de saisie des points d'interpolation.

Mode Automatique

- Définition d'un intervalle $[x_0, x_n]$ via des champs numériques.
- Choix du nombre de points n (de 2 à 500 via un curseur).
- Les coordonnées y_i sont générées aléatoirement dans [-5, 5].
- Un bouton **Réinitialiser / Regénérer** permet de tirer un nouveau jeu de points.

Mode Manuel

- Saisie directe des listes x et y séparées par des virgules (ex. : 0, 1, 2, 3).
- Validation en temps réel avec retour d'erreur si les tailles ne concordent pas.

Mode Multi-Problèmes

- Génération simultanée de k problèmes (de 2 à 500), chacun avec un nombre de points croissant (n, n+1, n+2, ...).
- Permet l'analyse statistique des performances sur une série de cas.

3.2 Paramètre de Répétition (N_ITER)

Un curseur **N_ITER** (de 1 à 20) contrôle le nombre de répétitions utilisées par *timeit* pour mesurer le temps de calcul moyen de chaque méthode. Cela améliore la fiabilité des mesures en réduisant la variance due aux aléas du système.

---

4. Méthode 1 — Interpolation de Lagrange

4.1 Principe Mathématique

L'interpolation de Lagrange consiste à construire un polynôme unique P(x) de degré ≤ n passant exactement par n+1 points (x_i, y_i).

```
L_i(x) = PRODUIT (j≠i) de [(x - x_j) / (x_i - x_j)]

P(x) = SOMME (j=0 à n) de [y_j × L_j(x)]
```

Propriété fondamentale : P(x_i) = y_i pour tout i (hypothèse H1).

4.2 Implémentation SymPy

- Calcule chaque base $L_i(x)$ de manière **symbolique exacte**.
- Construit et simplifie le polynôme $P(x)$ développé.
- Retourne les bases individuelles et le polynôme sous forme symbolique.
- **Avantage :** expression mathématique lisible et vérifiable analytiquement.
- **Inconvénient :** plus lent à cause des manipulations algébriques.

4.3 Implémentation SciPy

- Utilise *scipy.interpolate.lagrange* pour construire un objet poly1d.
- Calcul purement **numérique** basé sur les coefficients flottants.
- **Avantage :** exécution très rapide, vectorisable directement.
- **Inconvénient :** perte de la lisibilité symbolique ; instabilité numérique possible pour un grand nombre de points (phénomène de Runge).

4.4 Vérifications Affichées

Hypothèse	Condition vérifiée
H1	$P(x_i) = y_i$ pour tout i (interpolation exacte aux noeuds)

---

5. Méthode 2 — Spline Quadratique

5.1 Principe Mathématique

La spline quadratique remplace le polynôme global par des **polynômes du second degré par morceaux** $S_i(x)$ définis sur chaque intervalle $[x_i, x_{i+1}]$.

La construction repose sur les dérivées aux noeuds $z_i = S'(x_i)$, calculées par récurrence :

```
z_0 = 0
```

```
z_{i+1} = [2(y_{i+1} - y_i) / h_i] - z_i où h_i = x_{i+1} - x_i
```

Chaque morceau est alors :

```
S_i(x) = [(z_{i+1} - z_i) / (2 h_i)] × (x - x_i)² + z_i(x - x_i) + y_i
```

5.2 Implémentation SymPy

- Calcule les z_i symboliquement.
- Construit chaque morceau S_i(x) comme expression SymPy.
- Assemble une fonction Piecewise pour l'évaluation globale.
- Retourne les expressions, les noeuds et la fonction globale.

5.3 Implémentation SciPy

- Utilise *calculer_zi* pour calculer les dérivées numériquement (récurrence sur NumPy).
- Construit un objet PPoly (polynôme par morceaux) de SciPy.
- Évaluation rapide et vectorisée sur tout tableau de points.

5.4 Vérifications Affichées

Hypothèse	Condition vérifiée
H1	S(x_i) = y_i — interpolation exacte aux noeuds
H2	S_i(x_{i+1}) = S_{i+1}(x_{i+1}) — continuité aux raccords
H3	S'_i(x_{i+1}) = S'_{i+1}(x_{i+1}) — continuité de la dérivée première

---

6. Comparaison des Performances

Pour chaque méthode, l'application mesure et affiche :

- Le **temps moyen** d'exécution SymPy et SciPy (en millisecondes).
- Le **ratio SymPy / SciPy** : facteur d'accélération de SciPy.

- Un **graphique en barres** comparant les deux temps.

En mode multi-problèmes, des courbes d'évolution montrent comment les temps progressent avec la taille des problèmes.

Méthode	SymPy	SciPy	Observations
Lagrange	Lent (algèbre symbolique)	Rapide	Ratio typique : 10× à 100×
Spline	Lent (calcul par morceaux)	Rapide	Ratio similaire

---

7. Visualisation

Pour chaque méthode, un graphique superpose :

- La courbe **SymPy** (trait plein, violet).
- La courbe **SciPy** (trait pointillé, bleu ou vert).
- Les **points donnés** (x_i, y_i) en rouge.
- Pour la spline : des **lignes verticales grises** marquant les noeuds de raccord.

Les deux courbes doivent coïncider visuellement, confirmant la cohérence des deux implémentations.

---

8. Bibliothèques Utilisées

Bibliothèque	Rôle
streamlit	Interface web interactive
numpy	Calcul numérique vectorisé

sympy	Calcul symbolique exact
scipy	Interpolation numérique optimisée
matplotlib	Génération des graphiques
timeit	Mesure des temps d'exécution

---

9. Points Forts et Limites

Points Forts

- Interface claire et pédagogique, adaptée à un cours d'analyse numérique.
- Double vérification des hypothèses mathématiques (H1, H2, H3).
- Comparaison directe symbolique / numérique illustrant le compromis lisibilité / performance.
- Support du mode multi-problèmes pour une analyse de complexité empirique.

Limites

- L'instabilité numérique de Lagrange pour un grand n (phénomène de Runge) n'est pas signalée explicitement.
- La condition $z_0 = 0$ n'est qu'une des conditions aux limites possibles pour la spline ; les splines cubiques offrent un meilleur comportement en pratique.
- Le mode mono-problème génère des y_i sans graine fixe, limitant la reproductibilité des tests.

---

10. Conclusion

Cette application constitue un outil pédagogique complet pour illustrer les enjeux de l'interpolation numérique. Elle démontre concrètement que si le calcul symbolique (SymPy) offre une transparence mathématique précieuse pour la compréhension et la vérification, le

calcul numérique (SciPy) s'impose en pratique par sa rapidité et sa scalabilité.

L'étudiant peut ainsi expérimenter librement sur des jeux de données variés et observer en temps réel les compromis inhérents à chaque approche, renforçant ainsi sa compréhension théorique par l'expérimentation directe.